

统一代理接入第三方模型技术手册

Claude Code 与 Codex 通过本机 HTTPS 代理统一接入 OpenAI-compatible / Anthropic-compatible 模型服务

版本日期：2026-05-14 | 密级建议：内部技术资料 | 敏感值均使用占位符

核心结论： Claude Code 与 Codex 可以通过同一个本机 HTTPS 代理接入第三方模型。Claude 使用槽位映射，Codex 使用真实模型名 profiles。

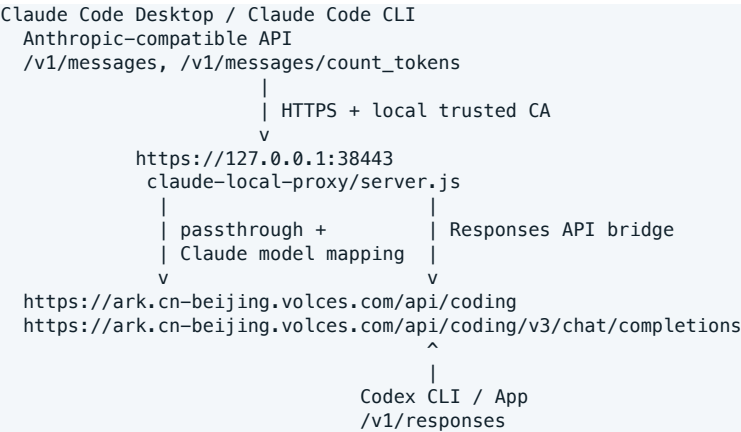
1. 目的与适用范围

- 本文定义一套标准方案：通过一个本机 HTTPS 统一代理，让 Claude Code Desktop/CLI 与 Codex CLI/App 同时接入第三方模型服务。
- 适用对象包括个人 Mac、公司内网 Mac、远程 SSH 管理的 macOS 机器，以及需要把 OpenAI-compatible 或 Anthropic-compatible 服务接入本地 AI 编程工具的场景。
- 本文使用火山 Ark coding endpoint 作为已验证示例；其他兼容服务可以沿用相同结构，只需要替换 upstream、模型名与认证方式。

2. 统一代理优势

- 单一入口：Claude Code 与 Codex 都指向本机 HTTPS endpoint，端口、证书、日志和启动方式统一管理。
- 清晰模型策略：Claude Code 侧保留 Claude 槽位名，Codex 侧直接使用真实模型名，避免两套工具互相影响。
- 协议差异内聚：Claude 的 Anthropic-compatible 请求与 Codex 的 Responses API 请求都在代理层适配，上游服务只承担模型推理。
- 验证口径一致：health、代理日志、Desktop main.log、Codex profile smoke test 与 tool call 构成统一验收矩阵。
- 安全边界清楚：API key 只进入本机配置或环境变量，不写入代理源码、文档、PPT、runbook 或 Git。

3. 目标架构



3.1 组件清单

组件	标准位置	职责
本机统一代理	<PROJECT_ROOT>/claude-local-proxy/server.js	唯一入口；监听 127.0.0.1:38443；处理 Claude 透传与 Codex 协议转换
本地证书	<PROJECT_ROOT>/claude-local-proxy/certs/ca.crt, server.crt, server.key	让 Desktop/Electron、CLI、host loop 都能信任本机 HTTPS
LaunchAgent	~/Library/LaunchAgents/com.cj.claude-local-https-proxy.plist	登录后自动启动代理；提供上游地址、模型映射和证书路径
Claude Desktop 3P config	~/Library/Application Support/Claude-3p/configLibrary/*.json	Desktop third-party inference 指向统一代理
Claude CLI settings	~/.claude/settings.json	Claude Code CLI/host 通过 ANTHROPIC_* 环境变量指向统一代理
Codex config	~/.codex/config.toml	Codex provider 使用 responses wire API，profiles 直接使用真实模型名
验证日志	<PROJECT_ROOT>/claude-local-proxy/logs/*.log, ~/Library/Logs/Claude-3p/main.log	排查入口健康、模型映射、协议转换与上游错误

3.2 代理路由

路径	方法	调用方	代理行为
/health	GET	运维健康检查	返回 ok、Claude upstream、Codex upstream、模型映射
/healthz	GET	轻量健康检查	返回 ok
*/responses	POST	Codex CLI/App	解析 Responses API，转换为 Chat Completions，再还原 Responses 输出
其他路径	任意	Claude Code Desktop/CLI	转发到 Anthropic-compatible upstream，并按 Claude 槽位映射模型

4. 模型策略

Claude Code 使用槽位模型名，Codex 使用真实模型名。这是本方案最重要的配置分界。

工具/配置	客户端模型名	实际上游模型	策略	建议用途
Claude Code	claude-opus-4-6	glm-5.1	通过代理映射	复杂推理/高质量输出
Claude Code	claude-sonnet-4-6	kimi-k2.6	通过代理映射	默认主力模型
Claude Code	claude-haiku-4-5	doubao-seed-2.0-pro	通过代理映射	快速/低成本任务
Codex ark-doubao	doubao-seed-2.0-pro	doubao-seed-2.0-pro	真实模型名	默认/快速任务
Codex ark-kimi	kimi-k2.6	kimi-k2.6	真实模型名	复杂编码任务
Codex ark-glm	glm-5.1	glm-5.1	真实模型名	高质量推理任务

5. 统一代理配置

LaunchAgent 或 shell 环境应提供以下变量。

```
LISTEN_HOST=127.0.0.1
LISTEN_PORT=38443
UPSTREAM_BASE_URL=https://ark.cn-beijing.volces.com/api/coding
CODEX_UPSTREAM_BASE_URL=https://ark.cn-beijing.volces.com/api/coding/v3
BIG_MODEL=glm-5.1
MIDDLE_MODEL=kimi-k2.6
SMALL_MODEL=doubao-seed-2.0-pro
TLS_CERT_FILE=<PROJECT_ROOT>/claude-local-proxy/certs/server.crt
TLS_KEY_FILE=<PROJECT_ROOT>/claude-local-proxy/certs/server.key
UPSTREAM_TIMEOUT_MS=300000
KEEP_ALIVE_TIMEOUT_MS=300000
HEADERS_TIMEOUT_MS=310000
```

5.1 代理必须实现的逻辑

1. Claude 分支：透传 /v1/messages、/v1/messages/count_tokens 等 Anthropic-compatible 请求，并把 Claude 槽位名改写为真实模型。
2. Codex 分支：接收 /v1/responses，转换成 /chat/completions，再把上游结果还原为 Responses API 输出。

6. 配置 Claude Code

6.1 Claude Desktop 3P Gateway

```
{
  "provider": "gateway",
  "gatewayBaseUrl": "https://127.0.0.1:38443",
  "gatewayApiKey": "<ARK_API_KEY>",
```

```

"gatewayAuthScheme": "bearer",
"inferenceModels": [
  { "id": "claude-sonnet-4-6", "name": "Sonnet 4.6" },
  { "id": "claude-opus-4-6", "name": "Opus 4.6" },
  { "id": "claude-haiku-4-5", "name": "Haiku 4.5" }
],
"hideAnthropicSignIn": true
}

```

6.2 Claude Code CLI / host settings

```

{
  "env": {
    "ANTHROPIC_BASE_URL": "https://127.0.0.1:38443",
    "ANTHROPIC_AUTH_TOKEN": "<ARK_API_KEY>",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "claude-opus-4-6",
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "claude-sonnet-4-6",
    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "claude-haiku-4-5",
    "NODE_USE_SYSTEM_CA": "1",
    "NODE_EXTRA_CA_CERTS": "<PROJECT_ROOT>/claude-local-proxy/certs/ca.crt",
    "SSL_CERT_FILE": "<PROJECT_ROOT>/claude-local-proxy/certs/ca.crt"
  }
}

```

配置要求：删除 ANTHROPIC_MODEL 和旧 modelOverrides；默认模型只写 Claude 槽位名。

6.3 Desktop host binary 与 Cowork

Desktop Code host 的版本目录以 Claude Desktop 日志中的 [CCD] Initialized with version ... 为准。Cowork 若报 SSL 证书失败，需要通过 launcher 注入 NODE_USE_SYSTEM_CA、NODE_EXTRA_CA_CERTS 与 SSL_CERT_FILE。

7. 配置 Codex

Codex 通过 provider + profiles 使用统一代理。base_url 指向 https://127.0.0.1:38443/v1，wire_api 使用 responses。

```

model_provider = "ark-coding"
model = "doubao-seed-2.0-pro"
model_reasoning_effort = "medium"
disable_response_storage = true

[model_providers.ark-coding]
name = "Volcengine Ark Coding via unified local proxy"
wire_api = "responses"
requires_openai_auth = true
base_url = "https://127.0.0.1:38443/v1"
supports_websockets = false

[profiles.ark-doubao]
model_provider = "ark-coding"
model = "doubao-seed-2.0-pro"
model_reasoning_effort = "medium"

[profiles.ark-kimi]
model_provider = "ark-coding"
model = "kimi-k2.6"
model_reasoning_effort = "high"

```

```
[profiles.ark-glm]
model_provider = "ark-coding"
model = "glm-5.1"
model_reasoning_effort = "high"
```

常用 profile 切换命令：

```
codex -p ark-doubao
codex -p ark-kimi
codex -p ark-glm
```

7.1 Codex CLI 与 Codex App 切换模型

Codex CLI 支持 -p/--profile；Codex App 更适合通过默认配置或启动参数覆盖模型。不要把 CLI profile 切换方式直接套到 App 上。

场景	操作方法	说明
Codex CLI	<code>`codex -p ark-kimi`</code>	读取 `[profiles.*]`，适合命令行临时切换。
Codex App 默认	改 <code>~/.codex/config.toml`</code> 顶层 <code>`model`</code> 后重启 App	最稳方式；新会话按默认模型启动。
Codex App 临时	<code>`codex app /path -c 'model="kimi-k2.6"' ...`</code>	启动 App 时覆盖配置；适合一次性打开特定 workspace。

Codex App 默认切换：修改 `~/.codex/config.toml` 顶层 `model_provider`、`model`、`model_reasoning_effort`，然后退出并重新打开 Codex App；新会话会使用新的默认模型。

Codex App 临时切换：

```
# 以 Kimi profile 对应模型打开某个 workspace
codex app /path/to/project \
  -c 'model_provider="ark-coding"' \
  -c 'model="kimi-k2.6"' \
  -c 'model_reasoning_effort="high"'

# 以 Doubao 打开某个 workspace
codex app /path/to/project \
  -c 'model_provider="ark-coding"' \
  -c 'model="doubao-seed-2.0-pro"' \
  -c 'model_reasoning_effort="medium"'

# 以 GLM 打开某个 workspace
codex app /path/to/project \
  -c 'model_provider="ark-coding"' \
  -c 'model="glm-5.1"' \
  -c 'model_reasoning_effort="high"'
```

注意：当前已打开会话不保证热切换模型。验证时新开会话或重启 App，再看代理日志中的 `codex responses model <客户端模型> -> <上游模型>`。

8. 验证与验收

检查项	命令/位置	通过标准
-----	-------	------

代理进程	launchctl print gui/\${id -u}/com.cj.claude-local-https-proxy	state = running
端口	ls -l -n -iTCP:38443 -sTCP:LISTEN	只有统一代理监听 38443 ; 旧 38444 不监听
健康检查	curl -sk https://127.0.0.1:38443/health	返回 ok、upstream、codexUpstream 与模型字段
Claude Desktop	~/Library/Logs/Claude-3p/main.log	ConfigHealth recomputed { state: 'healthy', provider: 'gateway' }
Claude 请求	代理日志	POST /v1/messages -> 200 ; 出现 claude-* -> 真实模型映射
Codex 默认	codex	默认 doubao-seed-2.0-pro 可回复
Codex profiles	codex -p ark-doubao / ark-kimi / ark-glm	三个 profile 均可回复
Tool call	Codex 中触发 shell/tool call	function_call 与 function_call_output 闭环正常

9. 运维与故障处理

故障	常见原因	处理方式
Desktop gateway unhealthy	Desktop 3P config base URL 错误、证书未信任或代理未运行	先查 /health、Keychain、main.log，再查代理日志
Claude 调错模型	保留了 ANTHROPIC_MODEL 或 modelOverrides，或请求模型名不是 Claude 槽位	删除强制模型字段，只保留槽位默认值；映射放在代理层
Codex 401/403	API key 未进入 Codex provider 或 Authorization 未透传	检查 ~/.codex/config.toml 与环境变量；不要把 key 写入 server.js
Codex tool call 失败	Responses API 与 Chat Completions 转换不完整	检查 function_call、function_call_output、tools、tool_choice 的转换日志
Cowork server is busy	UI 泛化错误，真实原因常是 SSL certificate verification failed	安装 claude-ca-launcher，给 host loop 注入 CA 环境
System keychain 写入失败	SSH 无交互授权被 macOS 拦截	在目标 Mac 本机交互式 sudo，或用 MDM/配置描述文件下发 CA

10. 回滚方案

- 3. 停止统一代理 LaunchAgent。
- 4. 恢复 server.js 备份。
- 5. 恢复 ~/.codex/config.toml 与 ~/.claude/settings.json 备份。
- 6. 如需恢复双代理模式，再加载旧 Codex LaunchAgent 并确认旧端口监听。
- 7. 回滚后分别验证 Claude /v1/messages、Codex /v1/responses 与 tool call。

11. 安全与交付边界

- 不在材料中保存真实 API key、SSH 密码、cookie、私钥或完整 authorization header。
- certs/*.key 不提交公共仓库；迁移机器时优先重新生成证书。
- 代理日志对外分享前必须扫描 authorization、x-api-key、api-key、cookie。
- 远程 SSH 密码在完成配置后建议替换为 SSH key 或轮换。
- 公司批量部署时，CA 信任优先通过 MDM/配置描述文件下发。